



VIT
BHOPAL



VITYARTHI PROJECT REPORT

Mobile Phone Usage Trends Explorer

A menu-driven data exploration and visualization application

Submitted by	Vir Mehta
Course	B.Tech Computer Science Engineering (Cyber Security & Digital Forensics)
Student ID	26BCY10048
Submission platform	VITyarthi
Report date	29 September 2026

This report provides details on the project scope, requirements, design, implementation, sample results, and testing approach.

1. Introduction

Mobile Phone Usage Trends Explorer is a menu-driven Python application that helps people inspect mobile phone user-count data by country, city, and year. It uses pandas for CSV and table operations and Matplotlib for line and bar charts. The project applies and demonstrates relevant concepts covered in B.Tech Computer Science Engineering (Cyber Security & Digital Forensics), including Python programming, tabular data operations, and visualization.

1.1 Problem statement

It can be challenging to understand how mobile phone usage changes over time. The project presents mobile-user data as charts and summaries that people can explore, supporting awareness of usage trends across locations and years.

1.2 Objectives

- Present mobile phone user counts in a form that is accessible to general users.
- Let users compare trends for selected cities and countries across the years in the dataset.
- Provide basic summaries and data operations through an interactive menu.
- Demonstrate course concepts in Python programming, tabular data processing, and visualization.

1.3 Scope

The current deliverable is a local command-line application. It reads the bundled **final.csv**, displays line charts, bar charts, and a comparison chart, prints record summaries, supports filters and other data manipulations, and saves modified records as CSV. It does not include a website, mobile application, live data feed, forecasting, user accounts, or a database server.

1.4 Target users

The target users are members of the general public interested in mobile phone usage trends, as well as students and teachers analyzing similar data. Users can compare city, country, and year and analyze the number of users in these locations.

1.5 Data overview

The supplied dataset contains 120 records, 6 countries, 30 cities, and four observation years: 2010, 2015, 2020, and 2025. It has four fields: Country, City, Year, and Users Millions. Each country contributes 20 records, and each city has four observations. The CSV contains no missing values or exact duplicate rows in the reviewed copy. Its original external source was not included with the project files.

2. Requirements

2.1 Functional requirements

- **Read and normalize data:** Read final.csv, trim header whitespace, and recognize the Users (Millions) column variant.
- **Menu workflow:** Provide a main menu with visualization, analysis, and manipulation submenus.
- **Select locations:** Let users choose countries and cities from numbered lists or search by text.
- **Display summaries:** Show the first ten records, top user-count records, and descriptive statistics.
- **Visualize trends:** Plot a city line chart, a city bar chart, and a comparison line chart for up to five cities.
- **Filter records:** Filter the dataset by country and inclusive year range.
- **Calculate growth:** Add Growth_Rate using pct_change within each city/country series.
- **Clean and sort:** Remove exact duplicate rows and sort by a selected column.
- **Export data:** Save the current_df selection to a user-named CSV file.

2.2 Non-functional requirements

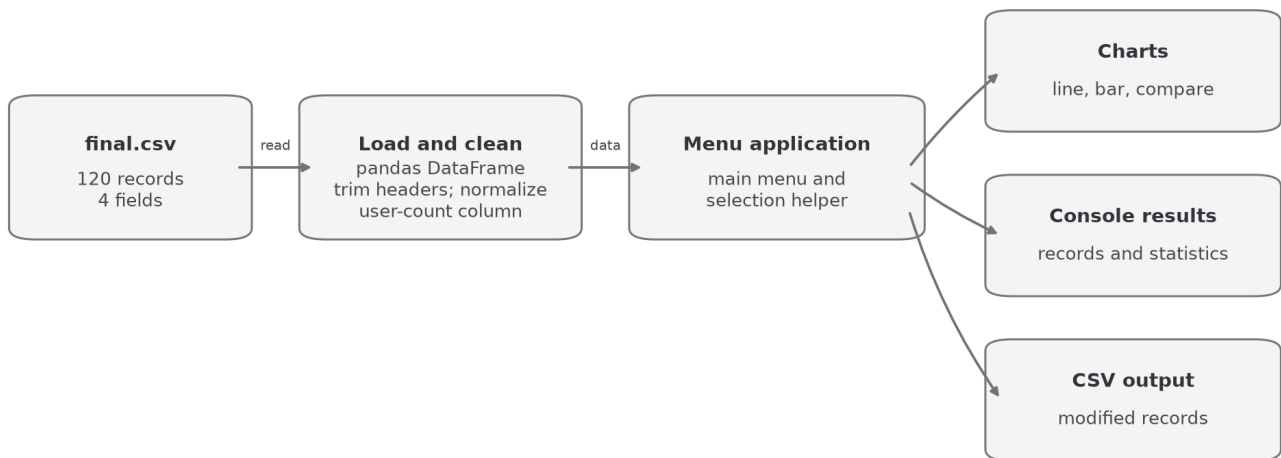
The status notes identify what the current code demonstrates and what remains a target or improvement.

- **Usability:** Use readable menu choices and label charts with city/country, year, and units. *Status:* Partly evidenced; menus and chart labels are present, but no user study was performed.
- **Performance:** On a standard lab computer, load the 120-row dataset and display a selected chart within five seconds. *Status:* Target only; no timing was collected.
- **Reliability:** Show helpful messages for invalid entries, missing files, and invalid columns. *Status:* Partly met; invalid menu and numeric input are handled, while file/schema validation is limited.
- **Data integrity:** Keep the source CSV unchanged and save modified data separately with confirmation. *Status:* Partly met; export is available, but choosing the source filename is not prevented.
- **Portability:** Resolve final.csv relative to the script, independent of the working directory. *Status:* Implemented with `pathlib.Path(__file__)`.

The performance target is a proposed acceptance criterion, not a measured result. Reliability and data-integrity improvements are listed under future enhancements.

3. System architecture

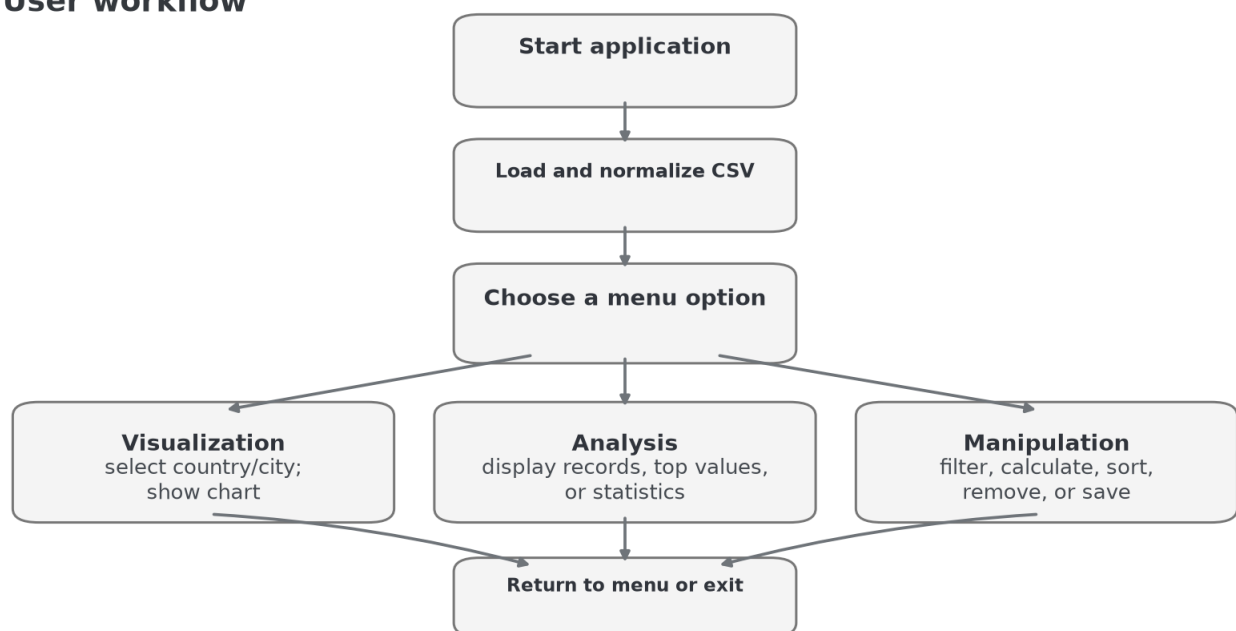
System architecture



The application has six logical functional areas: CSV loading and header normalization, selection/search, menu navigation, visualization, analysis, and data manipulation. These areas use functions inside `Final_Project.py` rather than separate source modules or classes.

4. Process flow

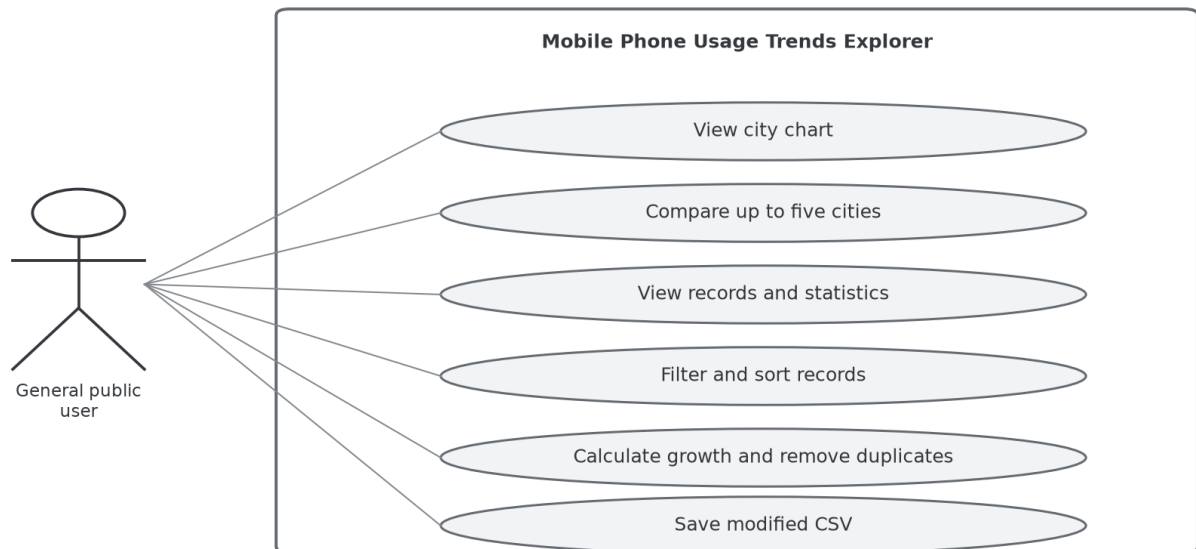
User workflow



5. Design diagrams

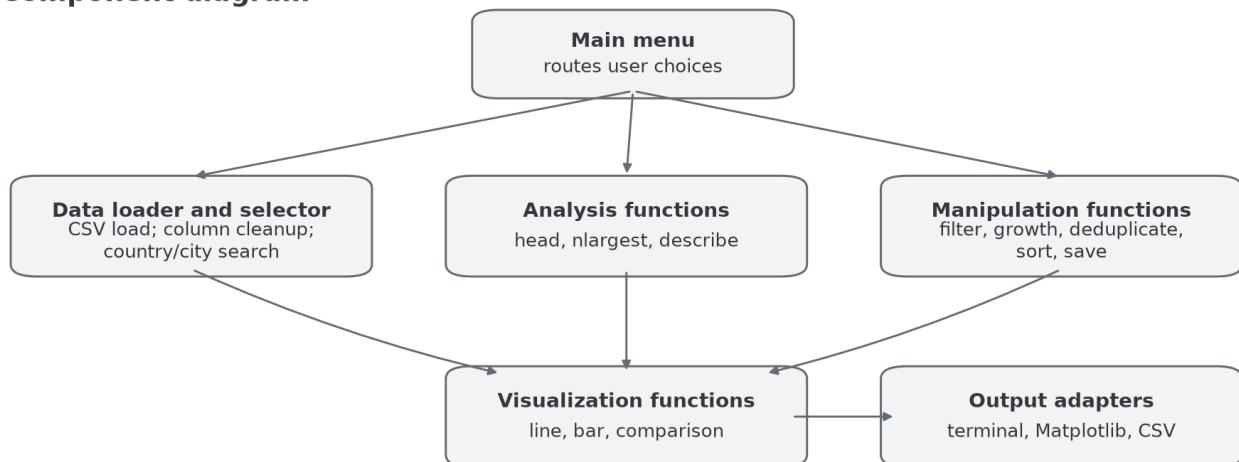
5.1 Use case diagram

Use case diagram



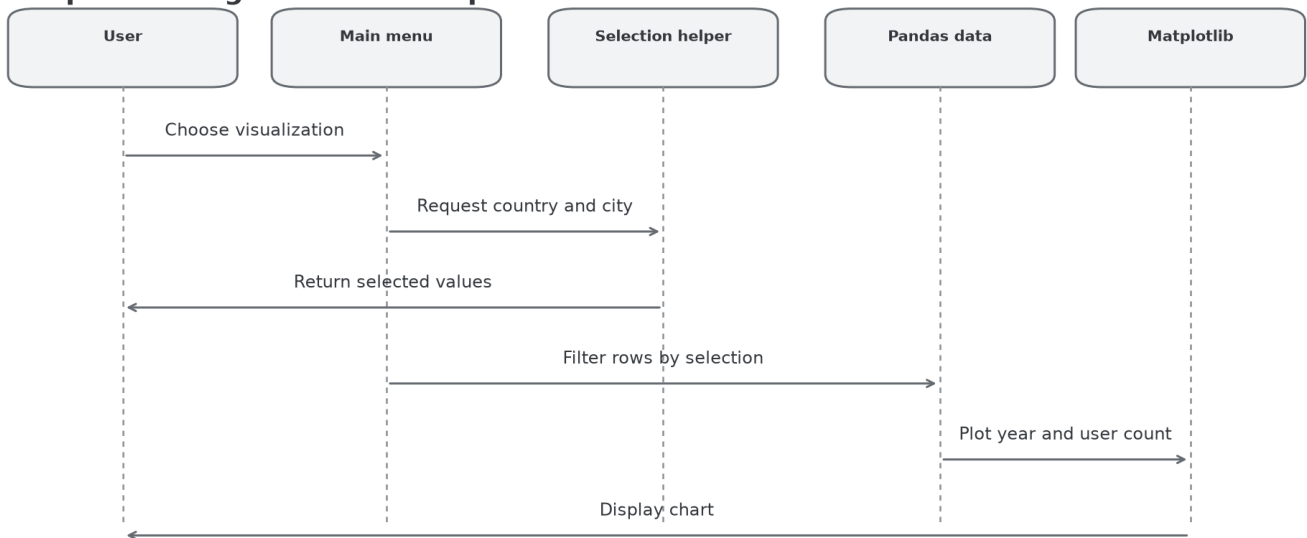
5.2 Component diagram

Component diagram



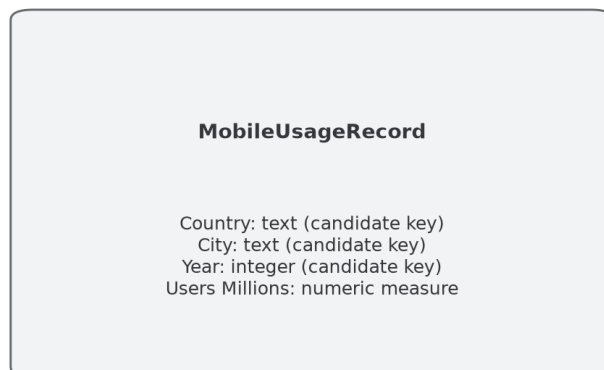
5.3 Sequence diagram

Sequence diagram - chart request



5.4 CSV storage schema

CSV storage schema (ER-style)



6. Data and design decisions

6.1 Dataset schema

- **Country:** Text; country for the observation.
- **City:** Text; city within the country.
- **Year:** Integer; observation year (2010, 2015, 2020, or 2025).
- **Users Millions:** Numeric; reported count in millions.

6.2 Design decisions and rationale

- **Command-line menu:** Keeps interaction straightforward for a small local course project without introducing a web stack.
- **pandas DataFrame operations:** Support CSV input, filtering, sorting, descriptive statistics, duplicate removal, and grouped percentage change. [2]
- **Matplotlib figures:** Provide line, bar, and multi-series comparison plots using project data. [3]
- **CSV storage:** Matches the flat tabular dataset and makes modified data easy to inspect or share.
- **Group growth by city and country:** Prevents comparisons between unrelated location series; observations are sorted by year first.

7. Implementation details

- **CSV load and normalization:** Top-level pandas read and header-cleanup operations load final.csv and normalize the user-count column.
- **Selection/search — select_from_list:** Shows numbered choices and supports text search.
- **Menu navigation — main_menu, visualization_menu, analysis_menu, manipulation_menu:** Routes the user through project features.
- **Visualization — plot_city_line_chart, plot_city_bar_chart, compare_cities_line_chart:** Filters selected locations and renders Matplotlib charts.
- **Analysis:** Shows rows, highest user-count records, and describe() statistics.
- **Data manipulation:** Filters, calculates Growth_Rate, removes duplicates, sorts, and exports records.

Growth-rate note: pct_change calculates change between consecutive observations. Because the dataset has five-year intervals, the current output is interval growth, not an annualized year-over-year rate. The Growth_Rate label should be clarified or the calculation annualized.

8. Results and screenshots

The dataset summary below was calculated from the supplied final.csv. The application smoke run also displayed pandas descriptive statistics for all 120 rows.

- **Records and fields:** 120 records and 4 fields.
- **Coverage:** 6 countries, 30 cities, and 4 years (2010, 2015, 2020, 2025).
- **Completeness:** No missing values in the four reviewed fields.
- **Exact duplicates:** 0 duplicate rows.
- **User-count range:** 6 to 57 million; mean 20.99 million across all observations.
- **Highest 2025 observation:** Beijing, China, with 57 million users.



Across this sample, the selected city series increase between the four recorded years. The figures describe the supplied dataset and should not be interpreted as an independently verified estimate of actual population-level mobile usage. The dataset provenance is not stated in the provided files.

9. Testing approach and evaluation

The project has no automated test suite. The following smoke check was run: start the application, open Analysis, choose Statistics, return to the main menu, and exit. The run completed successfully, recognized the Users (Millions) header, and reported 120 observations. The chart and data-manipulation features were also checked manually; these checks are separate from the smoke test described above.

- **Main menu and statistics path:** Menu opens, statistics print, and the user returns and exits cleanly. *Status:* Passed in smoke run.
- **Column-name normalization:** Users (Millions) is recognized and normalized to Users Millions. *Status:* Observed in smoke run.
- **Line, bar, and city comparison charts:** Selected city/year values and labels appear correctly. *Status:* Manually verified.
- **Filters, growth, duplicates, sorting, and save:** Output rows and saved CSV match the requested operations. *Status:* Manually verified.
- **Missing file/schema and invalid data:** Helpful errors appear without unhandled exceptions. *Status:* Not verified; validation should be strengthened.

9.1 Challenges and current limitations

- The input file uses the header Users (Millions), so the program must normalize that label before analysis.
- The data has four observations per city at five-year intervals; pct_change is not an annualized growth rate.
- Some manipulation choices, including duplicate removal and sorting, start from the original DataFrame rather than the current filtered selection, so sequential operations may not compose as users expect.
- The program checks some user input but does not fully validate required CSV columns, missing values, or numeric types before using them.
- The data source and collection methodology are not recorded in the supplied project files.

9.2 Learnings and key takeaways

- CSV data can be loaded into a DataFrame and explored with filters, sorting, grouped transformations, and descriptive statistics.
- Line and bar charts communicate changes over time; titles, units, legends, and year labels help users interpret comparisons.
- A menu-driven workflow can expose multiple analysis tasks in a single application.
- Data provenance, input validation, test coverage, and metric definitions are important parts of a trustworthy analysis tool.

10. Future enhancements

- Validate that the CSV exists and contains all required columns with numeric Year and Users Millions values; show helpful messages for invalid data.
- Keep a single working DataFrame through successive manipulation operations and prevent overwriting the original input unless explicitly requested.
- Annualize growth rates for the five-year observation interval or rename the output to interval growth.
- Add automated unit and validation tests for data loading, filters, growth calculations, sorting, duplicate handling, and export.
- Split the single source file into separate modules as the project grows, while preserving the current feature groups.
- Document the dataset source, collection method, and any assumptions about what a user count measures.
- Consider a graphical or web interface and accessibility improvements for broader public use.

11. References

VITyarthi Course

Dr. Velmurugan L

Informatics Practices with Python, Textbook for CBSE Class XII by Preeti Arora